

Air University Islamabad Department of Computer Science



LAB-1

SUBMITTED Sep, 2026

DSA Lab Mam. Shamsa Kanwal

Submitted By:

Imad Khan	2500623
-----------	---------

Lab Objectives:

By the end of this lab session, students will be able to:

1. Install and configure a C++ development environment (VS Code + GCC/MinGW) for the semester.
2. Explain what a data structure is and why choosing the right one matters for efficiency.
3. Trace and implement Linear Search and Binary Search algorithms.
4. Trace and implement Bubble Sort, Merge Sort, and Quick Sort.
5. Explain the structure of a Singly Linked List (SLL) and implement insertion, deletion, traversal, and search operations on it.
6. Independently solve and demonstrate the given lab tasks during the session.

LAB TASK

Task 1 — Linear Search

Write a C++ program that stores 10 integers in an array (values entered by the user), asks the user for a value to search, and uses Linear Search to report the index at which the value was found, or a “not found” message otherwise.

- **Input:** an array of 10 integers and a target value.
- **Output:** index of the target, or “Element not found”.
- **Requirement:** implement the search inside a separate function `linearSearch(int arr[], int n, int key)`.

CODE:

```
#include <iostream>
using namespace std;

int linearSearch(int arr[], int n, int key)
{
    for (int i = 0; i < n; i++)
    {
        if (arr[i] == key)
        {
            return i;
        }
    }

    return -1;
}

int main()
{
    int arr[10];
    int key;

    cout << "Enter 10 integers:\n";

    for (int i = 0; i < 10; i++)
    {
        cin >> arr[i];
    }

    cout << "Enter value to search: ";
    cin >> key;

    int result = linearSearch(arr, 10, key);
```

```

if (result != -1)
{
    cout << "Element found at index: " << result << endl;
}
else
{
    cout << "Element not found" << endl;
}

return 0;
}

```

Output

```

Enter 10 integers:
10
20
30
40
50
60
70
80
90
100
Enter value to search: 70
Element found at index: 6

```

Task 2 — Binary Search

Write a C++ program that sorts an array of 10 integers (you may reuse a sort from Task 3, or use a simple sort) and then performs Binary Search for a value entered by the user.

- **Requirement:** implement `binarySearch(int arr[], int n, int key)` iteratively (using a while loop, not recursion).
- Print how many comparisons the binary search made, and compare this to how many linear search would have made in the worst case.

CODE:

```

#include <iostream>
using namespace std;

void sortArray(int arr[], int n)
{

```

```

for (int i = 0; i < n - 1; i++)
{
    for (int j = 0; j < n - i - 1; j++)
    {
        if (arr[j] > arr[j + 1])
        {
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}

```

```

int binarySearch(int arr[], int n, int key, int &comparisons)
{
    int low = 0;
    int high = n - 1;

    comparisons = 0;

    while (low <= high)
    {
        int mid = (low + high) / 2;

        comparisons++;

        if (arr[mid] == key)
        {
            return mid;
        }
        else if (key > arr[mid])
        {
            low = mid + 1;
        }
        else
        {
            high = mid - 1;
        }
    }

    return -1;
}

```

```

int main()
{
    int arr[10];
}

```

```
int key;
int comparisons;

cout << "Enter 10 integers:\n";

for (int i = 0; i < 10; i++)
{
    cin >> arr[i];
}

sortArray(arr, 10);

cout << "Sorted array: ";

for (int i = 0; i < 10; i++)
{
    cout << arr[i] << " ";
}

cout << "\nEnter value to search: ";
cin >> key;

int result = binarySearch(arr, 10, key, comparisons);

if (result != -1)
{
    cout << "Element found at index: " << result << endl;
}
else
{
    cout << "Element not found" << endl;
}

cout << "Binary Search comparisons: "
    << comparisons << endl;

cout << "Linear Search worst-case comparisons: 10"
    << endl;

return 0;
}
```

```
Output

Enter 10 integers:
10
20
40
50
60
80
70
30
90
100
Sorted array: 10 20 30 40 50 60 70 80 90 100
Enter value to search: 90
Element found at index: 8
Binary Search comparisons: 3
Linear Search worst-case comparisons: 10
```

Task 3 — Bubble Sort

Implement Bubble Sort on an array of at least 10 integers entered by the user. Print the array after every full pass so the sorting process is visible step by step.

```
#include <iostream>
using namespace std;
```

CODE:

```
void bubbleSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

```
        cout << "Pass " << i + 1 << ": ";

        for (int k = 0; k < n; k++)
        {
            cout << arr[k] << " ";
        }

        cout << endl;
    }
}

int main()
{
    int arr[10];

    cout << "Enter 10 integers:\n";

    for (int i = 0; i < 10; i++)
    {
        cin >> arr[i];
    }

    cout << "\nBubble Sort Process:\n";

    bubbleSort(arr, 10);

    cout << "\nFinal sorted array: ";

    for (int i = 0; i < 10; i++)
    {
        cout << arr[i] << " ";
    }

    return 0;
}
```

Output

Bubble Sort Process:

Pass 1: 80 70 60 50 10 20 30 10 90 100

Pass 2: 70 60 50 10 20 30 10 80 90 100

Pass 3: 60 50 10 20 30 10 70 80 90 100

Pass 4: 50 10 20 30 10 60 70 80 90 100

Pass 5: 10 20 30 10 50 60 70 80 90 100

Pass 6: 10 20 10 30 50 60 70 80 90 100

Pass 7: 10 10 20 30 50 60 70 80 90 100

Pass 8: 10 10 20 30 50 60 70 80 90 100

Pass 9: 10 10 20 30 50 60 70 80 90 100

Final sorted array: 10 10 20 30 50 60 70 80 90 100

Task 4 — Merge Sort

Choose ONE of the two algorithms and implement it recursively on an array of at least 10 integers:

- **Merge Sort:** implement `mergeSort()` and a separate `merge()` helper function.
- **Quick Sort:** implement `quickSort()` and a separate `partition()` helper function.

Print the array before and after sorting.

CODE:

```
#include <iostream>
using namespace std;
```

```
class Node
{
public:
    int data;
    Node* next;

    Node(int value)
    {
        data = value;
        next = NULL;
    }
};
```

```
class SinglyLinkedList
{
```

private:

Node* head;

public:

SinglyLinkedList()

```
{  
    head = NULL;  
}
```

// Insert at end

void insert(int value)

```
{  
    Node* newNode = new Node(value);  
  
    if (head == NULL)  
    {  
        head = newNode;  
        return;  
    }  
  
    Node* temp = head;  
  
    while (temp->next != NULL)  
    {  
        temp = temp->next;  
    }  
  
    temp->next = newNode;  
}
```

// Display list

void display()

```
{  
    Node* temp = head;  
  
    while (temp != NULL)  
    {  
        cout << temp->data << " -> ";  
        temp = temp->next;  
    }  
  
    cout << "NULL" << endl;  
}
```

```

// 1. Insert at position
void insertAtPosition(int value, int pos)
{
    Node* newNode = new Node(value);

    if (pos == 1)
    {
        newNode->next = head;
        head = newNode;
        return;
    }

    Node* temp = head;

    for (int i = 1; i < pos - 1 && temp != NULL; i++)
    {
        temp = temp->next;
    }

    if (temp == NULL)
    {
        cout << "Invalid position" << endl;
        delete newNode;
        return;
    }

    newNode->next = temp->next;
    temp->next = newNode;
}

```

```

// 2. Count nodes
int countNodes()
{
    int count = 0;

    Node* temp = head;

    while (temp != NULL)
    {
        count++;
        temp = temp->next;
    }

    return count;
}

```

```

// 3. Find maximum value
int findMax()
{
    if (head == NULL)
    {
        cout << "List is empty" << endl;
        return -1;
    }

    int maxValue = head->data;

    Node* temp = head->next;

    while (temp != NULL)
    {
        if (temp->data > maxValue)
        {
            maxValue = temp->data;
        }

        temp = temp->next;
    }

    return maxValue;
}

// 4. Reverse list
void reverseList()
{
    Node* previous = NULL;
    Node* current = head;
    Node* nextNode = NULL;

    while (current != NULL)
    {
        nextNode = current->next;
        current->next = previous;
        previous = current;
        current = nextNode;
    }
    head = previous;
}

};
int main()
{
    SinglyLinkedList list;
    // Create list with 6 values

```

```
list.insert(10);
list.insert(20);
list.insert(30);
list.insert(40);
list.insert(50);
list.insert(60);

cout << "Original List:\n";
list.display();

// Insert at position
cout << "\nAfter inserting 25 at position 3:\n";

list.insertAtPosition(25, 3);

list.display();

// Count nodes
cout << "\nTotal number of nodes: "
    << list.countNodes() << endl;

// Find maximum
cout << "\nMaximum value: "
    << list.findMax() << endl;

// Reverse
cout << "\nBefore reverse:\n";
list.display();

list.reverseList();

cout << "After reverse:\n";
list.display();
return 0;
}
```

Output

Original List:

10 -> 20 -> 30 -> 40 -> 50 -> 60 -> NULL

After inserting 25 at position 3:

10 -> 20 -> 25 -> 30 -> 40 -> 50 -> 60 -> NULL

Total number of nodes: 7

Maximum value: 60

Before reverse:

10 -> 20 -> 25 -> 30 -> 40 -> 50 -> 60 -> NULL

After reverse:

60 -> 50 -> 40 -> 30 -> 25 -> 20 -> 10 -> NULL

Task 5 — Singly Linked List

Using the `SinglyLinkedList` class built in Part C as your starting point, add the following new member functions:

1. `insertAtPosition(int value, int pos)` — insert a new node at a given 1-based position.
2. `countNodes()` — return the total number of nodes currently in the list.
3. `findMax()` — return the largest data value stored in the list.
4. `reverseList()` — reverse the list in place (without creating a new list) so that head ends up pointing to what was previously the last node.

CODE:

```
#include <iostream>
using namespace std;
```

```
class Node
{
public:
    int data;
    Node* next;

    Node(int value)
    {
        data = value;
        next = NULL;
    }
};
```

```
class SinglyLinkedList
{
private:
    Node* head;

public:

    SinglyLinkedList()
    {
        head = NULL;
    }

    // Insert at end
    void insert(int value)
    {
        Node* newNode = new Node(value);

        if (head == NULL)
        {
            head = newNode;
            return;
        }

        Node* temp = head;

        while (temp->next != NULL)
        {
            temp = temp->next;
        }

        temp->next = newNode;
    }

    // Display list
    void display()
    {
        Node* temp = head;

        while (temp != NULL)
        {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
    }
}
```

```
    cout << "NULL" << endl;
}
```

```
// 1. Insert at position
```

```
void insertAtPosition(int value, int pos)
{
    Node* newNode = new Node(value);
```

```
    if (pos == 1)
    {
        newNode->next = head;
        head = newNode;
        return;
    }
```

```
    Node* temp = head;
```

```
    for (int i = 1; i < pos - 1 && temp != NULL; i++)
    {
        temp = temp->next;
    }
```

```
    if (temp == NULL)
    {
        cout << "Invalid position" << endl;
        delete newNode;
        return;
    }
```

```
    newNode->next = temp->next;
    temp->next = newNode;
}
```

```
// 2. Count nodes
```

```
int countNodes()
```

```
{
    int count = 0;
```

```
    Node* temp = head;
```

```
    while (temp != NULL)
    {
        count++;
        temp = temp->next;
    }
```

```

    }

    return count;
}

// 3. Find maximum value
int findMax()
{
    if (head == NULL)
    {
        cout << "List is empty" << endl;
        return -1;
    }

    int maxVal = head->data;

    Node* temp = head->next;

    while (temp != NULL)
    {
        if (temp->data > maxVal)
        {
            maxVal = temp->data;
        }

        temp = temp->next;
    }

    return maxVal;
}

```

```

// 4. Reverse list
void reverseList()
{
    Node* previous = NULL;
    Node* current = head;
    Node* nextNode = NULL;

    while (current != NULL)
    {
        nextNode = current->next;

        current->next = previous;

        previous = current;
    }
}

```

```
        current = nextNode;
    }

    head = previous;
}
};
```

```
int main()
{
    SinglyLinkedList list;

    // Create list with 6 values
    list.insert(10);
    list.insert(20);
    list.insert(30);
    list.insert(40);
    list.insert(50);
    list.insert(60);

    cout << "Original List:\n";
    list.display();

    // Insert at position
    cout << "\nAfter inserting 25 at position 3:\n";

    list.insertAtPosition(25, 3);

    list.display();

    // Count nodes
    cout << "\nTotal number of nodes: "
        << list.countNodes() << endl;

    // Find maximum
    cout << "\nMaximum value: "
        << list.findMax() << endl;

    // Reverse
    cout << "\nBefore reverse:\n";
```

```
list.display();

list.reverseList();

cout << "After reverse:\n";
list.display();

return 0;
}
```

Output

Original List:
10 -> 20 -> 30 -> 40 -> 50 -> 60 -> NULL

After inserting 25 at position 3:
10 -> 20 -> 25 -> 30 -> 40 -> 50 -> 60 -> NULL

Total number of nodes: 7

Maximum value: 60

Before reverse:
10 -> 20 -> 25 -> 30 -> 40 -> 50 -> 60 -> NULL
After reverse:
60 -> 50 -> 40 -> 30 -> 25 -> 20 -> 10 -> NULL

---END---